Estimate	Exact time	Time range	Query
8 points	4.5 days	4–7 days	devTime:[4 TO 7] AND storyPoints:8
4 points	2.25 days	2–4 days	devTime:[2 TO 4] AND storyPoints:4
2 points	1.125 days	1–2 days	devTime:[1 TO 2] AND storyPoints:2
1 point	.56 days	less than 1 day	devTime:[0 TO 1] AND storyPoints:1
Fibonacci series estimations with two-week sprints			
13 points	9	6–9 days	
8 points	5.53	4–6 days	
5 points	3.46	3–4 days	
3 points	2.07	2–3 days	
2 points	1.38	1–2 days	
1 points	.69	less than a day	

Note that in table 7.1 I've put the Fibonacci series of estimates along with the series of power of 2 estimates, another common estimation series. In the power of 2 series, each possible estimate is double the previous estimate.

This is helpful, but it would be better to have a scale that told you if your estimations are high or low and how off they are. For example, a 0 means that estimations meet the time equivalents, a number greater than 0 means your team is underestimating, and a value less than 0 means your team is overestimating. Using a scale like this has benefits:

- It's actionable.
- If you see your estimate health go under 0, you can start adding more time to your estimates to get them back to a level of accuracy.
- If you see your estimate health go over 0, you can start cutting your estimates a bit to get back to a better level of accuracy.
- It's easy to communicate.
- You can easily share this outside your team without a long explanation.
- It allows you to digest several data points at a glance.
- You can put this on your dashboards to keep everyone aware of the latest data.

To create a rating number you'll need to write an algorithm that figures out the time-estimate ratio of the estimate in question and checks it against the